Blockchain-Based Secure Deduplication and Shared Auditing in Decentralized Storage

Guohua Tian, Yunhan Hu, Jianghong Wei, Zheli Liu, Xinyi Huang, Xiaofeng Chen, *Senior Member, IEEE*, and Willy Susilo, *Fellow, IEEE*

Abstract—Data deduplication and public auditing are significant for providing secure and efficient network storage services. However, the existing data deduplication schemes supporting auditing not only cannot effectively alleviate the threats of the single point of failure and duplicate-faking attack, but also have to bear the massive waste of computation and storage resources caused by metadata redundancy and repetitive audit tasks. In this article, we propose a blockchain-based secure deduplication and shared auditing scheme in decentralized storage. Specifically, our scheme utilizes a novel deduplication protocol based on the double-server storage model to achieve efficient space-saving while protecting data users from losing data under a single point of failure and duplicate-faking attack. Besides, it sharply reduces the computation and storage costs of metadata by introducing a lightweight authenticator generation algorithm and update protocol. On this basis, our scheme further adopts a blockchain-based two-way shared auditing mechanism to achieve decentralized public auditing without the third-party auditor, in which the audit authenticators and results of outsourced data are shared among its users to avoid repetitive audit tasks. Security and performance analysis indicates the practicability of our scheme.

Index Terms—Data deduplication, shared auditing, blockchain, decentralized storage

1 Introduction

With the widespread popularity of network storage services, more and more data users tend to outsource their data to storage service providers (SSP) (e.g., cloud storage). As reported by IDC, the volume of global data will increase from 33 ZB in 2018 to 175 ZB in 2025 [1]. Despite the compelling benefits of network storage services, its scalability and security issues have received close attention.

On the one hand, the repetitive uploads of the same data lead to the fact that almost 75% of outsourced data stored in SSPs are duplicate copies [2]. To save storage resources, convergent encryption (CE) [3] is proposed to achieve encrypted deduplication [4], in which data users encrypt their data with its hash value, so that SSP can detect and delete the duplicate encrypted data. Inspired by CE, many improved schemes [5], [6], [7], [8], [9] have been proposed to enhance the availability of deduplication. Nevertheless, these schemes are still vulnerable to the single point of failure (SPOF) since only one copy of each data is stored in the storage server. Likewise, malicious users can also launch a duplicate-faking attack (DFA) to destroy the unique copy of the outsourced data, meaning that other users will lose their data once they upload it to the SSP.

On the other hand, users will lose physical control of their data after outsourcing it, which means that SSP may tamper with or delete the rarely accessed data to save storage space [10], [11]. To ensure data integrity, Provable Data Possession (PDP) [12] is introduced to achieve data auditing in a random challenge manner, in which users split their data into blocks and generate verification metadata (i.e., authenticators), which enables users to check their data by challenging SSP with partial data blocks selected randomly. After that, many schemes [13], [14], [15], [16], [17] have been proposed to improve the practicability of data auditing.

However, most existing public auditing schemes are impractical since they delegate the audit tasks of all users to a fully trusted third-party auditor (TPA) that is difficult to find in applications. To this end, Xu *et al.* [18] utilized a non-fully trusted TPA monitored by the blockchain [19] to achieve public auditing in data deduplication scenarios, which alleviates part of the burden of TPA by avoiding repetitive audit tasks. Likewise, Yuan *et al.* [20] used smart contract [21] to perform no-TPA public auditing in data deduplication scenario, whereas it will inevitably lead to expensive gas costs. Furthermore, most existing schemes

- *Guohua Tian, Yunhan Hu, and Xiaofeng Chen are with the State Key Laboratory of Integrated Service Networks (ISN), Xidian University, Xi'an, Shaanxi 710071, China, and also with the State Key Laboratory of Cryptology, Beijing 100878, China. E-mail: gh_tian0621@163.com, yhhu_1@stu.xidian.edu.cn, xfchen@xidian.edu.cn.*
- *Jianghong Wei is with the State Key Laboratory of Mathematical Engineering and Advanced Computing, Zhengzhou, Henan 450002, China. E-mail: jianghong.wei.xxgc@gmail.com.*
- *Zheli Liu is with the College of Cyber Science, Nankai University, Tianjin 300071, China. E-mail: liuzheli@nankai.edu.cn.*
- *Xinyi Huang is with the Fujian Provincial Key Laboratory of Network Security and Cryptology, College of Mathematics and Informatics, Fujian Normal University, Fuzhou, Fujian 350000, China. E-mail: xyhuang@fjnu.edu.cn.*
- *Willy Susilo is with the Institute of Cybersecurity and Cryptology, University of Wollongong, Wollongong, NSW 2522, Australia. E-mail: wsusilo@uow.edu.au.*

Manuscript received 14 Mar. 2021; revised 11 Sept. 2021; accepted 17 Sept. 2021. Date of publication 21 Sept. 2021; date of current version 11 Nov. 2022.
This work was supported in part by the National Nature Science Foundation of China under Grants 61960206014, 62032012, 62121001, and 62172434, in part by the Key Research and Development Program of Shaanxi under Grant 2020ZDLGY08-03, and in part by the Shandong Provincial Key Research and Development Program of China under Grant 2019JZZY020129.
(Corresponding author: Xiaofeng Chen.)
Digital Object Identifier no. 10.1109/TDSC.2021.3114160

have to divide the data into blocks $b_i \in \mathbb{Z}_p$ and generate audit authenticators, to prevent prover from pre-computing lightweight proof sources, which inevitably results in authenticators redundancy, especially in large file auditing. Therefore, a practical and secure storage service model combining data deduplication and auditing well is still desired.

1.1 Contributions

In this paper, we propose a blockchain-based secure data deduplication and shared auditing scheme in decentralized storage. Our main contributions are listed as follows:

1) We propose a novel deduplication protocol that uses a blockchain-based double-server storage model to protect data users from losing data under SPOF while achieving efficient space-saving. Besides, the proposed protocol not only enables subsequent uploader to upload data without encrypting the entire file, but also protects data users from losing their data under DFA.
2) We utilize the BLS signature to design a lightweight authenticator generation algorithm and the corresponding update protocol for reducing metadata redundancy, which is more suitable for achieving large file auditing.
3) We propose a blockchain-based two-way shared auditing mechanism to achieve public auditing without TPA, in which the entire auditing process will be shared among data users to avoid repetitive audit tasks. Besides, our scheme can also support batch auditing.

1.2 Related Work

In order to achieve privacy-preserving data deduplication, Douceur *et al.* [3] first proposed CE to enable each user of the same data to obtain the same ciphertext by encrypting the data with its hash value. On this basis, Bellare *et al.* [5] formalized the primitive message-locked encryption (MLE) for encrypted data deduplication and proposed three variants of CE: HCE1, HCE2, and RCE. Besides, they also defined a formal privacy model PRV$-CDA for MLE. Inspired by MLE, some deduplication schemes [6], [7], [8] focus on extending MLE into block-level data deduplication to seek more efficient space-saving, and other schemes [22], [23], [24] aim at improving the practicability of MLE-based deduplication by achieving various security or efficiency purposes.

Considering the SPOF in deduplication, a simple but expensive solution is to let data users store multiple copies of their data on different SSPs, which may lead to unacceptable storage fees for data users. Therefore, data users prefer to use a threshold secret sharing algorithm to split their data into blocks and then upload them to different SSPs to resist SPOF [24]. However, the distributed storage model brings more problems for integrity verification, especially in terms of latency and overheads. Thus, there is no doubt that if the storage cost of outsourced data is reduced to an acceptable manner, the multi-copy storage model will be more feasible to resist SPOF than the distributed storage approach.

In addition, the original client-side deduplication allows users to upload their data with its tag (e.g., a hash value), to save bandwidth. However, the malicious users can also acquire valid ownership by launching a proof-replay attack (PRA) with eavesdropped tags. To overcome this issue, Halevi *et al.* [25] proposed the notion of proofs-of-ownership (PoWs) and proposed a specific PoWs protocol based on Merkle Hash Tree. Moreover, since SSP cannot check the consistency of file tag and encrypted data, the malicious users might launch a duplicate-faking attack (DFA): upload an encrypted file $C' = Enc(F')$ with a tag $T = Tag(F)$. The subsequent users will lose their data F after uploading $C = Enc(F)$ with T. Although SSPs can adopt PoWs to detect DFA, there is no way to determine whether the DFA has occurred or is occurring. Thus, Wang *et al.* [26] introduced the traceable signature technique into data deduplication to trace the malicious users. Besides, Kim *et al.* [27] introduced a novel double-tag storage strategy to resist DFA. Unfortunately, their scheme is vulnerable to PRA since data users are allowed to prove their data possession with a tag. After that, Tian *et al.* [28] proposed an improved version based on ElGamal encryption. Nevertheless, both of them require each user to encrypt the entire data before uploading it, which is a common efficiency bottleneck of existing solutions in encrypted data deduplication.

For more reliable data outsourcing, Deswarte *et al.* [29] introduced a "challenge-response" model for verifying the integrity of outsourced data. On this basis, Ateniese *et al.* [12] first proposed the concept of PDP to achieve efficient remote data auditing in a random manner. Later, Wang *et al.* [30] proposed a public verifiable PDP scheme that employs a fully trusted TPA to reduce the audit burden on users. Concerning the recovery of corrupted data, Juels *et al.* [13] proposed a novel primitive called Proof-of-retrievability (PoR) based on the sentinel technique. After that, Shacham and Waters [14] proposed an improved PoR scheme by introducing erasure coding and BLS signature [31] techniques.

Considering the integrity verification in data deduplication, Zheng *et al.* [32] proposed proof-of-storage with deduplication (POSD) that combines client-side deduplication and data auditing to save storage space while ensuring data integrity. However, this trivial combination model will inevitably result in a rapid increase of data tags and authenticators. Yuan *et al.* [33] proposed an improved POSD that supports batch processing with fixed costs. Inspired by MLE, Liu *et al.* [34] proposed a message-locked integrity auditing and deduplication scheme, which requires users to generate authenticators with convergent key, to achieve the deduplication over audit authenticators. Xu *et al.* [18] introduced a novel combination model, in which audit authenticators not only be used by TPA to check data integrity, but also be utilized by SSP to achieve the PoWs process.

Nevertheless, most existing TPA-aided public auditing schemes still face two problems: strong trust placed on TPA and heavy audit burden. Benefit from blockchain [19], [21], [35], [36], Zhang *et al.* [37] proposed a blockchain-based auditing scheme, which requires TPA to publish audit results on the blockchain such that data users can discover malicious TPA. Further, Xu *et al.* [18] proposed a

blockchain-based scheme that employs a not fully trusted TPA to implement data auditing under the supervision of the blockchain. For achieving auditing without TPA, Xu *et al.* [38] proposed a private auditing scheme based on arbitrable smart contract, in which all interaction information between server and user is recorded on the blockchain for fair arbitration. Furthermore, Yuan *et al.* [20] proposed a no-TPA public auditing scheme, which uses smart contracts to execute audit tasks and fair arbitration. In short, almost all of these schemes treat the blockchain as a simple black box.

The remainder of this paper is organized as follows. First, we introduce the problem formulation and construction of our scheme in Sections 2 and 3, respectively. Then, we analyze its security in Section 4 and performance in Section 5. Finally, the conclusion is drawn in Section 6.

2 Problem Formulation

In this section, we introduce the system model, definitions, security model of the proposed scheme.

2.1 System Model

The decentralized storage system model of the proposed scheme involves the following four types of entities:

- *Storage service provider (SSP)*: Honest-but-curious storage node, ranging from commercial organizations like Dropbox to individuals with redundant storage resources, provides paid data storage services for data users, which means that SSPs will perform various tasks honestly and learn data knowledge as much as possible. In particular, we assume that SSPs do not collude with each other, such that the only two physical data copies can be separately stored on two SSPs, and one of them can work as an auditor to audit the other one (auditee).
- *User (U)*: The user node who outsources his file F to storage nodes. When F has not been stored on any SSP of the decentralized storage system, U is called an initial uploader and uploads F to two SSPs selected according to service requirements. Otherwise, U is a subsequent uploader and uploads F to its two specific SSPs. Besides, U can not only employ one of them as an auditor to audit the other one, but also help SSPs to update the audit authenticators.
- *System manager (SM)*: The operational management team, similar to Bitcoin and Ethereum, is responsible for initializing the decentralized storage system.
- *Blockchain*: A public distributed ledger system with various nodes (i.e., storage nodes, user nodes, and miners), in which miners process transactions or execute smart contracts to earn rewards and drive the decentralized storage system.

2.2 Definitions

The proposed scheme, building on the variant algorithm $\mathsf{HCE2} = \mathsf{KeyGen}, \mathsf{Enc}, \mathsf{TagGen}, \mathsf{Dec}$[5] of symmetric encryption, consists of the following five algorithms (*Setup*, *KeyGen*, *Encrypt*, *TagGen*, *Decrypt*) and three interactive protocols (*Upload*, *Audit*, *AuthUpdate*).

1) *Setup*(λ) $\rightarrow$ $Para$: On input a security parameter λ, this algorithm outputs a set of system parameters $Para$.
2) *KeyGen*: On input file $F = m_1 \parallel \cdots \parallel m_n$, the key generation algorithm outputs the block keys $k_1, k_2, \ldots, k_n$ and file key pair (sk, pk) as follows:
   - *B-KeyGen*(m_i) $\rightarrow$ k_i : $\mathsf{HCE2.KeyGen}$ takes block m_i as input ($1 \le i \le n$), returns the block key k_i.
   - *F-KeyGen*(F) $\rightarrow$ (sk, pk) : takes the file F as input, returns the file secret key sk and the corresponding public key pk (the first file tag $t = pk$).
3) *Encrypt*: On input data blocks $\{m_i\}$ and keys $\{k_i\}$ for $1 \le i \le n$, the encryption algorithm outputs encrypted file $C = \{c_i\}$ and key ciphertext Ck as follows:
   - *B-Encrypt*(k_i, m_i) $\rightarrow$ c_i: $\mathsf{HCE2.Enc}$ takes a data block m_i and its key k_i as inputs ($1 \le i \le n$), returns encrypted block c_i.
   - *Bk-Encrypt*($sk, \{k_i\}_{1 \le i \le n}$) $\rightarrow$ Ck : takes file secret key sk and block keys $\{k_i\}_{1 \le i \le n}$ as inputs, returns the ciphertext of block keys Ck.
4) *TagGen*: On input all encrypted data blocks $\{c_i\}$, the tag generation algorithm outputs the block tags $\{T_i\}$ and the second file tag t^* as follows:
   - *B-TagGen*(c_i) $\rightarrow$ T_i : $\mathsf{HCE2.TagGen}$ takes block c_i as input ($1 \le i \le n$), returns its block tag T_i.
   - *F-TagGen*($\{T_i\}$) $\rightarrow$ t^* : takes all block tags $\{T_i\}$ as inputs, returns the second file tag t^*.
5) *Upload*: According to whether the file F exists in storage system, U executes one of the following subprotocols:
   - *Upload.ini*. If F does not exist, an initial upload protocol is executed as follows:
     - *AuthGen*(sk, c_i) $\rightarrow$ σ_i : For $1 \le i \le n$, takes sk and c_i as inputs, the authenticator generation algorithm outputs the authenticator σ_i. By running *B-KeyGen*, *Encrypt*, *TagGen*, and *AuthGen* algorithms, U obtains and uploads some necessary metadata $<T, \sigma, Ck>$ to SSP.
     - SSP searches for the unique blocks with tags $T = \{T_i\}$ locally, and requires U to return the corresponding blocks.
     - SSP checks the validity of blocks uploaded by U, and stores the necessary metadata.
   - *Upload.sub*. If F exists, a subsequent upload (i.e., PoW) protocol will be executed, which consists of the following three algorithms:
     - *PoW.Chal*($Para$) $\rightarrow$ $Chal_x$: takes $Para$ as input, this algorithm generates a PoW challenge nonce $Chal_x$ for deduplication.
     - *PoW.Proof*($Chal_x, \{c_x\}$) $\rightarrow$ $Proof_x$: based on $Chal_x$, this algorithm takes the challenged blocks $\{c_x\}$ as inputs, returns a proof $Proof_x$.
     - *PoW.Verify*($\{\sigma_x\}, Proof_x$) $\rightarrow$ $0/1$: based on $Proof_x$ and its aggregate authenticators $\{\sigma_x\}$, this algorithm outputs the verification result.
6) *Audit*: The interactive protocol realizes the data integrity auditing via the following three algorithms:

- *Aud.Chal*$(Para) \rightarrow Chal_y$: takes $Para$ as input, this algorithm generates a challenge nonce $Chal_y$ for data integrity auditing.
- *Aud.Proof*$(Chal_y, \{c_y\}) \rightarrow Proof$: based on $Chal_y$, this algorithm takes the challenged blocks $\{c_y\}$ as inputs, returns the integrity proof $Proof_y$.
- *Aud.verify*$(\{\sigma_y\}, Proof_y) \rightarrow 0/1$: based on the corresponding authenticators $\{\sigma_y\}$ and $Proof_y$, this algorithm outputs the audit result.

7) *AuthUpdate*: A concrete authenticator update protocol consists the following three algorithms:
   - *Update.PreCompute*$(c_i, s_i) \rightarrow \rho_i$: For $1 \leq i \leq n$, takes c_i and the random challenge seed s_i as inputs, the pre-computation algorithm returns the corresponding half-finished authenticator ρ_i.
   - *Update.Sign*$(sk, \rho_i) \rightarrow \sigma_i$: For $1 \leq i \leq n$, takes sk and ρ_i as inputs, the signature algorithm returns the corresponding authenticator σ_i
   - *Update.Verify*$(\{\rho_i\}, \{\sigma_i\}) \rightarrow 0/1$: takes $\{\rho_i\}$ and $\{\sigma_i\}$ as inputs, outputs verification result.
8) *Decrypt*: On input sk, Ck, and C, the decryption algorithm returns the plaintext $F = m_1 \parallel \cdots \parallel m_n$ through the following two sub-algorithms:
   - *Bk-Decrypt*$(sk, Ck) \rightarrow \{k_i\}$: HCE2.Dec takes sk and Ck as inputs, returns block keys $\{k_i\}$.
   - *F-Decrypt*$(k_i, c_i) \rightarrow m_i$: For $1 \leq i \leq n$, takes k_i and c_i as inputs, returns plaintext block m_i.

2.3 Security Model

Inspired by Shacham and Waters's work [14], we briefly define a security model for the proposed scheme.

- *Initialization*: A challenger $\mathcal{C}$ first generates a random file F and runs *KeyGen*, *Encrypt* algorithms to generate file key pair (sk, t) and encrypted blocks $\{c_i\}$. Then, $\mathcal{C}$ sends $Para = (t, \{c_i\})$ to adversary $\mathcal{A}$.
- *Query*: The adversary $\mathcal{A}$ randomly picks a block $c_j \in \{c_i\}$ and queries $\mathcal{C}$ for the corresponding hash value and signature until the query times up to q_s.
- *Challenge*: $\mathcal{C}$ checks $\mathcal{A}$ with a random challenge $Chal_t$ consisting of some blocks that have not been queried. According to $Chal_t$, $\mathcal{A}$ generates the corresponding aggregate signatures σ_t and proof $Proof_t$, and then returns them to $\mathcal{C}$.
- *Verify*: $\mathcal{C}$ checks whether σ_t is consistent with $Proof_t$. If so, $\mathcal{A}$ wins, and fails otherwise.

Definition 1. *For a random challenge Chal, any PPT adversary cannot pass the consistency verification of the proposed scheme with a non-negligible probability.*

For enhancing data privacy, we also set the following security goals for the proposed scheme.

- *Data privacy*. The proposed scheme should prevent both the malicious SSPs and users from obtaining the plaintext of the outsourced data.
- *Data integrity*. The proposed scheme should prevent the malicious adversary from launching a DFA, and allows data users to check the integrity of the outsourced data through data auditing or tag checking.
- *PRA resistance*. Both the PoW protocol and data auditing mechanism of the proposed scheme should prevent malicious adversaries (e.g., users or SSP) from a launching proof-replay attack.
- *SPOF resistance*. The proposed scheme should protect users from losing data caused by SPOF.

3 The Proposed Scheme

This section describes the proposed blockchain-based secure deduplication and shared auditing scheme in detail.

3.1 Overview

As depicted in Fig. 1, we use blockchain to establish a novel double-server storage model in decentralized storage, instead of treating it as a simple black box. On this basis, by integrating double-copy storage and novel client-side deduplication protocol, the data users except for the initial uploader can achieve data upload without encrypting the entire file, and all the outsourced data are protected from SPOF and DFA. In addition, we endow SSPs with the role of an auditor to check the other one (i.e., auditee) who stores the same outsourced data and thus achieve two-way shared auditing based on the blockchain without any TPA. Meanwhile, other users can learn data integrity from the public blockchain without repetitive auditing. Notably, the double-server storage model and two-way shared auditing enable auditors to periodically update their audit authenticators with the help of valid users, which allows users to use the hash values of data blocks to generate authenticators without considering the potential pre-computation attacks.

3.2 Concrete Structure

3.2.1 System Setup

Based on security parameter λ, *SM* runs the *Setup*(λ) algorithm to initialize a decentralized storage system. First, two multiplicative cyclic groups $\mathbb{G}_1$ and $\mathbb{G}_2$ with order p are selected to build a bilinear map e: $\mathbb{G}_1 \times \mathbb{G}_1 \rightarrow \mathbb{G}_2$. Second, a generator g and a random element u are selected from $\mathbb{G}_1$, two hash functions $H_1 : \{0,1\}^* \rightarrow \mathbb{Z}_p^*$ and $H_2 : \{0,1\}^* \rightarrow \mathbb{G}_1$ are defined to generate some necessary metadata, and two pseudo-random functions $\pi_1 : \{1, 2, \ldots, n\} \times \mathbb{Z}_p^* \rightarrow \{1, 2, \ldots, n\}$ and $\pi_2 : \{1, 2, \ldots, n\} \times \mathbb{Z}_p^* \rightarrow \mathbb{Z}_p^*$ are defined to assist user or SSP to generate the index and coefficient of challenged blocks. Besides, a uniform file chunking strategy is defined to enable users of the same file to obtain the same blocks. Finally, *SM* publishes the system parameters $Para = \{\mathbb{G}_1, \mathbb{G}_2, e, p, g, u, H_1, H_2, \pi_1, \pi_2\}$ on the blockchain.

3.2.2 Data Upload and Deduplication

When a user U wants to outsource a file $F = m_1 \parallel ... \parallel m_n$, U runs *F-KeyGen* algorithm to calculate file secret key $sk = H_1(F)$ and the first file tag $t = g^{sk}$ (file public key). Then, U searches the blockchain for the duplicates of tag t. If no duplicates exist, U will execute an *Initial Upload* process. Otherwise, U performs a *Subsequent Upload* process with two specific SSPs where the duplicate data F is stored.

Initial Upload. For the unique file F, U selects two suitable SSP_1 and SSP_2 according to his service demands (e.g.,

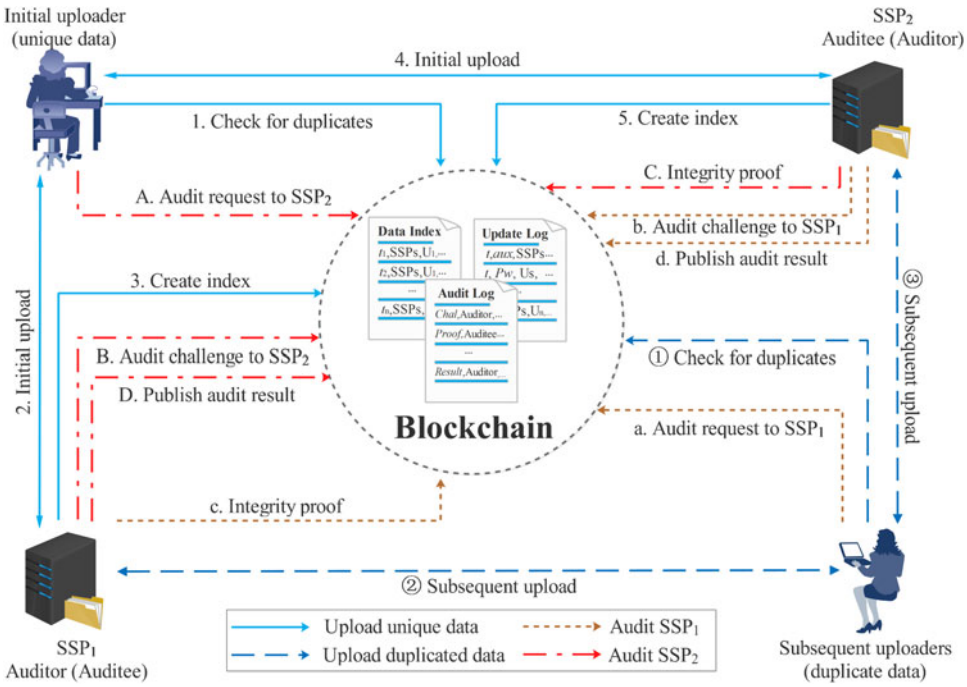


Fig. 1. The system model and workflow of the proposed scheme.

security ranks, storage capacity, etc), and then executes the initial upload protocol *Upload.ini* with them, respectively.

First, U runs the *B-KeyGen*, *Encrypt*, *B-TagGen* and *Auth-Gen* algorithms to generate block key set $K = \{k_i\}$, block ciphertext set $C = \{c_i\}$, block tag set $T = \{T_i\}$, and authenticator $\sigma = \{\sigma_i\}$ as follows.

$$\begin{cases} k_i & = H_1(m_i) \\ c_i & = \mathsf{HCE2.Enc}(k_i, m_i) \\ T_i & = H_1(c_i) \\ M_i & = H_2(t \parallel i) \\ \sigma_i & = (M_i u^{T_i})^{sk} \end{cases} \quad 1 \le i \le n. \tag{1}$$

Then, U generates the ciphertext of block keys as follow.

$$Ck = \mathsf{HCE2.Enc}(sk, k_1 \parallel k_2 \parallel ... \parallel k_n). \tag{2}$$

Second, U executes the following procedures:

1) U sends the upload request $Upload.ini = < t, T, \sigma, Ck >$ to SSP (SSP$_1$ or SSP$_2$) via a secure off-chain channel.
2) SSP looks up its local storage for duplicate $T_i \in T$, and obtains the tag set T_{dep} of duplicate block set C_{dep} and T_{uni} of unique block set C_{uni}, respectively. Then, SSP requests U to return the encrypted data blocks of C_{uni}.
3) Upon receiving the C_{uni} sent by U, SSP checks the consistency between unique blocks and their tags. If pass, SSP runs *F-TagGen* algorithm to generate the second file tag $t^* = H_1(T_1 \parallel ... \parallel T_n)$. Besides, SSP stores file F by maintaining the tuple $< t, t^*, T, Ck, \sigma >$ and C_{uni} locally. [1] Finally, SSP publishes a data index table $<t, t^*, U, SSP>$ on the blockchain for achieving deduplication in the subsequent upload phase.
4) U deletes the local file, and only maintains the tuple $<t, t^*, sk>$ for data access and retrieval.

Subsequent Upload. For the duplicate file F, U will perform a subsequent upload protocol *Upload.sub* with SSP$_1$ and SSP$_2$, respectively. More details are shown as follows:

1) Upon receiving the upload request $Upload.sub = < t >$ sent by U, SSP (SSP$_1$ or SSP$_2$) runs *PoW.Chal* algorithm to generate an ownership challenge $Chal_x = (z, r_1, r_2)$, where $z \in [1, n]$ and $r_1, r_2 \in \mathbb{Z}_p^*$ are three random numbers. Then, SSP sends $Chal_x = (z, r_1, r_2)$ to U through an off-chain channel.
2) Refer to $Chal_x$, U runs this algorithm *PoW.Proof* to calculate $a_i = \pi_1(i, r_1), b_i = \pi_2(i, r_2)$ for each $i \in [1, z]$. Then, this algorithm calculates the corresponding integrity proof by equation (3).

$$\begin{cases} k_{a_i} & = H_1(m_{a_i}) \\ c_{a_i} & = \mathsf{HCE2.Enc}(k_{a_i}, m_{a_i}) \\ \mu_i & = b_i H_1(c_{a_i}) \end{cases} \quad 1 \le i \le z \tag{3}$$

Finally, U generates the ownership proof $\mu = \{\mu_i\}$, or $\mu = \sum_i^z \mu_i$ for short, and returns it to SSP.
3) SSP runs the *PoW.Verify* algorithm to check the aggregate proof $Proof_x = \sum_{i=1}^z \mu_i$ as follows.

$$e\left(\prod_{i=1}^{z} \sigma_{a_i}^{b_i}, g\right) = e\left(\left(\prod_{i=1}^{z} M_{a_i}^{b_i}\right) u^{Proof_x}, t\right) \tag{4}$$

If Equation (4) holds, SSP regards U as a valid data user, and sends the second file tag t^* and access link to U. Otherwise, SSP aborts this process, which means that either the uploader is not a real data owner, or the data has been corrupted by DFA.

1. Note that the C_{dep} of outsourced file F has been stored during the previous initial upload of other files.

Considering the resistance against DFA, we allow the real data users who fail in *Upload.sub* process to execute the *Upload.ini* protocol. Based on the double-tag storage strategy, the proposed scheme binds each outsourced data with its two tags derived from plaintext and ciphertext, which enables the SSPs to check the consistency between the ciphertext and its tags, and thus protects the data users from losing data caused under duplicate-faking attack.

Correctness: we analyze the correctness of subsequent upload protocol *Upload.sub* as follows.

$$\begin{aligned} e\Big(\prod_{i=1}^{z}\sigma_{a_i}^{b_i}, g\Big) &= e\Big(\prod_{i=1}^{z}(M_{a_i}u^{T_{a_i}})^{b_i sk}, g\Big) \\ &= e\Big(\prod_{i=1}^{z}(M_{a_i}u^{H_1(c_{a_i})})^{b_i}, g^{sk}\Big) \\ &= e\Big(\Big(\prod_{i=1}^{z}M_{a_i}^{b_i}\Big)u^{\sum_{i=1}^{z}\mu_i}, g^{sk}\Big) \end{aligned} \tag{5}$$

Distinctly, iff $Proof_x = \sum_{i=1}^{z}\mu_i$ and $t = g^{sk}$, we get

$$e\Big(\prod_{i=1}^{z}\sigma_{a_i}^{b_i}, g\Big) = e\Big(\Big(\prod_{i=1}^{z}M_{a_i}^{b_i}\Big)u^{Proof_x}, t\Big) \tag{6}$$

The correctness of the protocol *Upload.sub* is demonstrated by Equations (5) and (6).

3.2.3 Shared Auditing

For more available data auditing, we allow users to employ SSP_2 (SSP_1) as an auditor to check the integrity of data F stored in auditee SSP_1 (SSP_2), so as to achieve two-way efficient data auditing without fully trusted TPA. Specifically, the auditor executes the *Audit* protocol with the auditee periodically, and then publishes all the audit results on the blockchain for achieving shared auditing.

1) The auditor runs *Audit.Chal* algorithm to generate a challenge $Chal_y = (z, r_1, r_2)$. First, this algorithm selects three random numbers $z \in [1, n]$ and $r_1, r_2 \in \mathbb{Z}_p^*$, and calculates the indexes $a_i = \pi_1(i, r_1)$ and coefficients $b_i = \pi_2(i, r_2)$ for challenged data blocks. Then, the auditor aggregates the authenticators of challenged blocks.

$$\sigma_y = \prod_{i=1}^{z}\sigma_{a_i}^{b_i} \tag{7}$$

To avoid repetitive effort, each data block will be checked only once in each audit period, and the remaining unchecked data blocks constitute a new challenged data block set for the next data audit. Finally, the auditor publishes $Chal_y$ on the blockchain.

2) After getting $Chal_y$ from blockchain, the auditee runs *Audit.Proof* algorithm to generate an integrity proof. In details, this algorithm computes the indexes $\{a_i\}_{1\le i\le z}$ and coefficients $\{b_i\}_{1\le i\le z}$ of the challenged blocks by π_1 and π_2, and generates the integrity proof $\mu = \{\mu_i\}$.

$$\mu_i = b_i H_1(c_{a_i}), i \in [1, z]. \tag{8}$$

Then, the auditee publishes the aggregated integrity proof $Proof_y = \sum_{i=1}^{z}\mu_i$ on the blockchain and sends μ to the auditor via an off-chain channel.

3) Upon receiving the integrity proof $Proof_y$ from the blockchain, the auditor executes *Audit.Verify* algorithm to check the validity of $Proof_y$ as follows.

$$e(\sigma_y, g) = e\left(\left(\prod_{i=1}^{z}M_{a_i}^{b_i}\right)u^{Proof_y}, t\right). \tag{9}$$

If Equation (9) holds, this algorithm outputs $result = 1$ to indicate that the auditee passes the auditing. Otherwise, $result = 0$, indicating that the checked data stored in the auditee is corrupted. Finally, the auditor shares the audit result $< \sigma_y, Proof_y, result >$ on the blockchain.

Correctness: Due to the same verification mechanism, the correctness proof of *Upload.sub* can also work for *Audit*.

3.2.4 Retrieval

When U wants to retrieve the outsourced file F, he shows $<t, t^*>$ to SSPs and downloads $< Ck, C >$. Then, U runs the *Decrypt* algorithm to calculate plaintext F:

$$\begin{cases} k_1 \parallel ... \parallel k_n &= \mathsf{HCE2.Dec}(sk, Ck) \\ m_i &= \mathsf{HCE2.Dec}(k_i, c_i), \quad 1 \le i \le n \end{cases} \tag{10}$$

Finally, U gets the file $F = m_1 \parallel ... \parallel m_n$. Notably, if U does not trust the integrity evidenced by audit results published on the blockchain, or there are some disputes occur in data auditing, our scheme allows U to verify the data integrity of F by downloading it and checking whether the equation $t^* = H_1(H_1(c_1) \parallel ... \parallel H_1(c_n))$ holds.

3.3 Authenticator Update

To prevent provers from pre-computing block hash values for data auditing (i.e., lightweight proof sources), we proposed an authenticator update protocol for our two-way shared auditing mechanism, which requires auditors to periodically update authenticators as follows.

1) SSP selects a random number $s \in \mathbb{Z}_p$, and calculates the challenge seed set $S = \{s_i\}_{1\le i\le n}$, where $s_i = H_1(i \parallel s)$. Then, SSP executes the *Update.PreCompute* algorithm to compute the set of half-finished authenticators $\rho = \{\rho_i\}$ and the auxiliary information aux by equation (11).

$$\begin{cases} \rho_i &= M_i u^{H_1(c_i \| s_i)} \\ aux &= \prod \rho_i \end{cases} \quad 1 \le i \le n \tag{11}$$

Then, SSP publishes $<t, t^*, aux>$ on blockchain, and sends ρ to a valid user U via a secure off-chain channel.

2) U first checks the consistency between ρ and aux. If pass, U runs the *Update.Sign* algorithm to compute the authenticator for each data block by:

$$\sigma_i = \rho_i^{sk} = (M_i u^{H_1(c_i \| s_i)})^{sk}, 1 \le i \le n \tag{12}$$

Then, U calculates a work proof $P_w = aux^{sk}$, and publishes (t, t^*, P_w, SSP) on the blockchain, which will be used as a work proof for U to get rewards.

Besides, the new authenticator set $\sigma = \{\sigma_i\}$ will be sent to SSP through a secure off-chain channel.

3) Upon obtaining the new authenticators, SSP executes *Update.verify* algorithm to check the consistency between P_w, σ, and aux as follows.

$$\begin{cases} P_w & = \prod_{i=1}^{n} \sigma_i \\ e(P_w, g) & = (aux, t) \end{cases} \tag{13}$$

If Equation (13) holds, SSP pays rewards to U and charges all users for data storage and auditing. Then, he uses $\sigma = \{\sigma_i\}$ to start a new audit period.

3.4 Batch Auditing

Next, we take batch auditing as an example to show the new workflow after updating authenticators. Suppose U wants to check the integrity of multiple files $\{F_1, F_2, \ldots, F_d\}$ at once, he performs an update protocol with the auditor to generate authenticators for batch auditing as follows.

1) Refer to the batch auditing request $\{(t_i, t_i^*)\}_{1 \le i \le d}$, the auditor picks a random number $s \in \mathbb{Z}_p^*$, and calculates the challenge seeds $S = \{s_{i,j}\}$, half-finished authenticators $\rho = \{\rho_{i,j}\}$ and auxiliary information aux.

$$\begin{cases} s_{i,j} & = H_1(i \parallel j \parallel s) \\ M_{i,j} & = H_2(t_i \parallel j) \\ \rho_{i,j} & = M_{i,j} u^{H_1(c_{i,j} \parallel s_{i,j})} \\ aux & = \prod(\prod \rho_{i,j}) \end{cases} \quad 1 \le i \le d, 1 \le j \le n \tag{14}$$

Finally, the auditor publishes $(\{t_i, t_i^*\}_{1 \le i \le d}, aux)$ on the blockchain, and sends ρ to U via an off-chain channel.

2) Upon receiving ρ, U checks the consistency between ρ and aux by Equation (14). If pass, U aggregates the secret keys of the checked files into $k_{mas} = sk_1 + sk_2 + \cdots + sk_d$, and computes the authenticator for each data block by signing the half-finished authenticator with k_{mas}, which is detailed as the following:

$$\sigma_{i,j} = \rho_{i,j}^{k_{mas}} = (M_{i,j} u^{H_1(c_{i,j} \parallel s_{i,j})})^{k_{mas}} \tag{15}$$

Besides, U generates a work proof $P_w = aux^{k_{mas}}$ and publishes $(\{t_i\}_{1 \le i \le d}, P_w, SSP)$ on the blockchain. Finally, all authenticators $\sigma = \{\sigma_{i,j}\}$ will be sent back to the auditor via a secure off-chain channel.

3) Before accepting $\sigma = \{\sigma_{i,j}\}$ as the authenticators of the next audit period, the auditor checks the consistency between σ, work proof P_w, and auxiliary information aux published on the blockchain, as follows.

$$\begin{cases} P_w & = \prod_{i=1}^{d} \left(\prod_{j=1}^{n} \sigma_{i,j} \right) \\ e(P_w, g) & = (aux, \prod_{i=1}^{d} t_i) \end{cases} \tag{16}$$

If the Equation (16) holds, the auditor accepts σ and starts the next new batch audit period.

Subsequently, the auditor periodically executes a batch auditing protocol to check the integrity of $\{F_1, F_2, \ldots, F_d\}$ stored in the auditee, and all the audit results are published on the blockchain. More details are shown as follows.

1) The auditor runs *Audit.Chal* algorithm to generate a audit challenge $Chal_y = (z, r_1, r_2, S^*)$, in which this algorithm selects three random numbers $z \in [1, nd]$ and $r_1, r_2 \in \mathbb{Z}_p^*$, and calculates the corresponding indexes and coefficients as follows.

$$\begin{cases} a_{v1} & = \pi_1(v, r_1) \bmod d \\ a_{v2} & = \pi_1(v, r_2) \bmod n \\ b_v & = \pi_2(v, r_2) \end{cases} \quad 1 \le v \le z. \tag{17}$$

Then, the auditor generates an aggregate authenticator σ_y and the challenge seed set $S^* = \{s_{a_{v1}, a_{v2}}\}$.

$$\sigma_y = \prod_{v=1}^{z} \sigma_{a_{v1}, a_{v2}}^{b_v}. \tag{18}$$

Finally, the challenge $Chal_y = (z, r_1, r_2, S^*)$ will be published on the blockchain.

2) Based on $Chal_y$, the auditee executes *Audit.Proof* algorithm to generate an integrity proof. In details, this algorithm computes the indexes $\{(a_{v1}, a_{v2})\}$ and coefficients b_v of the challenged data blocks by π_1 and π_2, and then generates the integrity proof by:

$$\mu_i = b_v H_1(c_{a_{v1}, a_{v2}} \parallel s_{a_{v1}, a_{v2}}), v \in [1, z]. \tag{19}$$

The auditee publishes the aggregate proof $Proof_y = \sum_z^{i=1} \mu_i$ on the blockchain and sends $\mu = \{\mu_i\}$ to the auditor via an off-chain channel.

3) Upon receiving the integrity proof $Proof_y$ from the blockchain, the auditor executes *Audit.Verify* algorithm to check whether the Equation (20) holds.

$$e(\sigma_y, g) = e\left(\left(\prod_{v=1}^{z} M_{a_{v1}, a_{v2}}^{b_v} \right) u^{Proof_y}, \prod_{i=1}^{d} t_i \right). \tag{20}$$

This algorithm outputs $result = 1$ iff the Equation (20) holds. Otherwise, $result = 0$. Finally, the audit result $<\sigma_y, Proof_y, result>$ is published on the blockchain.

4 Security Analysis

In this section, we analyze the security of the proposed scheme according to the aforementioned security goals.

Theorem 1. *As long as the underlying signature scheme used for audit authenticators in the proposed scheme is existentially unforgeable, DL and CDH problems are hard in bilinear group, any PPT adversary, in the random oracle model, cannot pass the verification of the proposed scheme with non-negligible probability during PoWs, data auditing, and authenticator update phases.*

Proof. We prove this theorem in a series of games.

Game 0. The initial game *Game 0* is a challenge game defined in Section 4.3, which is lossless in public verification.

Game 1. This game is the same as *Game 0*, except with one difference that the challenger $\mathcal{C}$ maintains a list of all file tags ever issued. If the adversary $\mathcal{A}$ submits a tag that not exists in the file tag list, $\mathcal{C}$ aborts and declares failure.

Game 2. This game is the same as *Game 1*, excepts with one difference that $\mathcal{C}$ keeps a list of its responses to Hash and signature St query made by $\mathcal{A}$. With aid of this list, $\mathcal{C}$ observes each instance of our verification protocol. If $\mathcal{A}$ succeed in any instance but his aggregate signature σ_t is not equal to $\prod_{(i,b_i)\in Chal_t} \sigma_i^{b_i}$ (where $Chal_t$ is the challenge issued by the verifier and $\{\sigma_i\}$ are the signature on the challenged file blocks in current instance), $\mathcal{C}$ aborts and declares failure.

Furthermore, we will establish some necessary notions for analyzing the difference in success probabilities between *Game 1* and *Game 2*. First, we assume the file that causes the failure consists of n blocks, has generating element $u \in \mathbb{G}_1$ and block signatures issued by St. Besides, suppose $Chal_t = \{(i, b_i)\}$ is the query that causes the failure and $\mathcal{C}$'s response to $Chal_t$ was $\{\mu_i'\}$ together with σ_t'. Let the expected response generated by an honest prover be $\{\mu_i\}$ and σ_t, where $\mu_i = b_i H_1(c_i)$ and $\sigma_t = \prod_{(i,b_i)\in Chal_t} \sigma_i^{b_i}$. Based on the correctness of the proposed scheme, the expected response satisfies the verification equation, i.e., that

$$e(\sigma_t, g) = e(\prod H_2(t \parallel i)^{b_i} u^{\mu_i}, t). \tag{21}$$

From the failure declared by $\mathcal{C}$, we get that $\sigma_t \neq \sigma_t'$ but σ_t' also satisfies the verification equation, i.e., that

$$e(\sigma_t', g) = e(\prod H_2(t \parallel i)^{b_i} u^{\mu_i'}, t), \tag{22}$$

where $t = g^{sk}$ is the public key and first tag of the challenge file. Observe that if $\mu_i = \mu_i'$ for each i, it follows from the verification equation that $\sigma_t = \sigma_t'$, which contradicts our assumption above. Therefore, if we define $\Delta\mu_i = \mu_i' - \mu_i$ for each $i \in Chal_t$, no doubt that an least one of $\Delta\mu_i$ is nonzero. Based on this conclusion, we further prove that we can construct a simulator $\mathcal{S}$ to solve the CDH problem if there is a non-negligible difference between the success probability of $\mathcal{A}$ in *Game 1* and *Game 2*.

The simulator $\mathcal{S}$ takes values $g, g^{sk}, h \in \mathbb{G}$ as inputs and tries to output h^{sk}. Specifically, $\mathcal{S}$ behaves like $\mathcal{C}$ in *Game 1*, with the following differences:

- In initialization, $\mathcal{S}$ sets the public key t as g^{sk} received in the challenge, which means that $\mathcal{S}$ does not know the corresponding secret key sk.
- $\mathcal{S}$ models the hash function H_2 as a random oracle that keeps a list of queries and responses to answer consistently. In a query instance, it chooses a random $r \leftarrow \mathbb{Z}_p$ and responses with $g^r \in \mathbb{G}$.
- When asked to store an encrypted file consists of the n blocks $\{c_i\}$, $\mathcal{S}$ behaves as below. First, $\mathcal{S}$ selects random values $\alpha, \beta \leftarrow \mathbb{Z}_p$, such that $u = g^{\alpha} h^{\beta}$. Then, $\mathcal{S}$ chooses a random value $r_i \leftarrow \mathbb{Z}_p$ and programs the random oracle a i as:

$$H_2(t \parallel i) = g^{r_i} / (g^{\alpha H_1(c_i)} \cdot h^{\beta H_1(c_i)}). \tag{23}$$

Now, $\mathcal{S}$ can calculate σ_i as follows:

$$\begin{aligned} \sigma_i &= (H_2(t \parallel i) u^{H_1(c_i)})^{sk} \\ &= (g^{r_i} / (g^{\alpha} \cdot h^{\beta})^{H_1(c_i)} \cdot (g^{\alpha} \cdot h^{\beta})^{H_1(c_i)})^{sk} \\ &= (g^{sk})^{r_i}. \end{aligned} \tag{24}$$

- $\mathcal{S}$ continues interacting with $\mathcal{A}$ until the special condition of *Game 2* occurs: $\mathcal{A}$ passes the verification with σ_t' other than the expected aggregate signature σ_t. Now, we can further get the following equation:

$$\begin{aligned} e(\sigma_t'/\sigma_t, g) &= e(\prod u^{\Delta\mu_i}, t) \\ &= e(\prod (g^{\alpha} h^{\beta})^{\Delta\mu_i}, t). \end{aligned} \tag{25}$$

Rearranging terms yields, we obtain:

$$e(\sigma_t' \cdot \sigma_t^{-1} \cdot t^{-\sum \alpha\Delta\mu_i}, g) = e(h, t)^{\sum \beta\Delta\mu_i}. \tag{26}$$

Note that $t = g^{sk}$, we see that we have found the solution to the CDH problem, that,

$$h^{sk} = (\sigma_t' \cdot \sigma_t^{-1} \cdot t^{-\sum \alpha\Delta\mu_i})^{\frac{1}{\sum \beta\Delta\mu_i}}. \tag{27}$$

Therefore, if there is a non-negligible difference between the success probability of $\mathcal{A}$ in *Game 1* and *Game 2*, we can construct a simulator $\mathcal{S}$ to employ $\mathcal{A}$ to solve CDH problem.

Game 3 This game is the same as *Game 2*, excepts with one difference that $\mathcal{C}$ tracks St queries and observes the interactive instances. This time, if in any of these instances $\mathcal{A}$ passes the verification but at least one of the aggregate messages μ_i is not equal to the expected $b_i H_1(c_i)$. Based on the same notations setting as *Game 2*, we further prove that we can construct a simulator $\mathcal{S}$ to solve the DL problem if there is a non-negligible difference between the success probability of $\mathcal{A}$ in *Game 2* and *Game 3*.

Here, $\mathcal{S}$ takes value $g, h \in \mathbb{G}$ as inputs and aims at outputting x such that $h = g^x$. Specifically, $\mathcal{S}$ behaves like $\mathcal{C}$ in *Game 2*, with the following differences:

- When asked to store an encrypted file consists of the n blocks $\{c_i\}$, $\mathcal{S}$ behaves as below. First, $\mathcal{S}$ selects random values $\alpha, \beta \leftarrow \mathbb{Z}_p$, such that $u = g^{\alpha} h^{\beta}$.
- $\mathcal{S}$ continues interacting with $\mathcal{A}$ until the special condition defined in *Game 3* occurs: $\mathcal{A}$ passes the verification with $\{\mu_i'\}$ other than the expected $\{\mu_i\}$. This time, we know that $\sigma_t = \sigma_t'$ and further obtain:

$$e(\sigma_t, g) = e(\sigma_t', g). \tag{28}$$

From which we conclude that

$$e(\sigma_t'/\sigma_t, g) = e(\prod u^{\Delta\mu_i}, t) \tag{29}$$

$$1 = \prod u^{\Delta\mu_i} = g^{\sum \alpha\Delta\mu_i} \cdot h^{\sum \beta\Delta\mu_i}. \tag{30}$$

TABLE 1
Comparisons of Functionality

	Data auditing				Data deduplication			
Schemes	Batch	No TPA	Shared	Lightweight	Privacy	No Data Loss	Model	Granularity
Scheme [18]	√	×	√	×	×	×	Client-side	File-level
Scheme [20]	√	√	×	×	√	×	Sever-side	Block-level
Our scheme	√	√	√	√	√	√	Client-side	Block-level

Now, we have found the solution to DL problem,

$$h = g^{-\frac{\sum \alpha \Delta\mu_i}{\sum \beta \Delta\mu_i}}. \tag{31}$$

Therefore, if there is a non-negligible difference between the success probability of $\mathcal{A}$ in *Game 2* and *Game 3*, we can construct a simulator $\mathcal{S}$ to employ $\mathcal{A}$ to solve DL problem. □

Theorem 2. *The proposed scheme can guarantee the privacy of outsourced data against malicious SSPs and users.*

Proof. Benefit from the underlying MLE (HCE2) encryption scheme [5], the proposed scheme provides outsourced data with PRV$-CDA level security as Bellare et al's claim in [5]. Since HCE2 algorithm can guarantee the indistinguishability between the ciphertexts of two unpredictable messages, the proposed scheme prevents not only SSP but also malicious adversary from retrieving plaintext even if they have obtained the encrypted data illegally. □

Theorem 3. *The proposed scheme can guarantee the integrity of outsourced data and protect data users from losing data under duplicate-faking attacks.*

Proof. Note that the proposed scheme provides triple integrity protection for outsourced data. First, the proposed deduplication protocol requires SSPs to store each data with its two tags, in which one of them is derived from plaintext and another one derived from the ciphertext. Since SSPs can verify the consistency between data and its second tag, the DFAs launched by malicious users will be invalidated. Second, the proposed scheme achieves blockchain-based shard auditing, which enables verifiable sharing of audit authenticators and results among the data users of the same data. Thus, data users can obtain the integrity information of outsourced data from the blockchain at any time. Finally, with the tag checking of HCE2, the proposed scheme allows users to check data integrity during data retrieval if they do not trust the audit results published on the blockchain. □

Theorem 4. *The proposed scheme can prevent malicious adversaries from obtaining valid ownership via a proof-replay attack.*

Proof. The proposed scheme utilizes the audit authenticators to achieve PoWs during the subsequent upload phase. By using a random challenge model, our scheme ensures the freshness of ownership challenge nonce and thus invalidates the potential PRAs. Meanwhile, malicious adversaries cannot obtain any useful information from the eavesdropped "challenge-response" instances to forge valid proof. Furthermore, the proposed scheme requires the auditors to update the audit authenticators by executing an interactive protocol with a valid data user periodically, which makes it harder for adversaries to launch a proof-replay attack. □

Theorem 5. *The proposed scheme can protect outsourced data from a single point of failure.*

Proof. With the aid of blockchain, the proposed scheme achieves more efficient space-saving and global resource optimization, which reduces the storage service fees to an acceptable manner for data users. On this basis, the double-server storage model of our scheme is more feasible than previous solutions in avoiding single point of failure. □

5 Performance

In this section, we evaluate the performance of the proposed scheme and the latest works [18], [20] in terms of functionality, theoretical analysis, and implementation.

5.1 Functionality Comparisons

As is shown in Table 1, we list the functionalities of compared schemes reflected in data auditing and deduplication. Notably, the notion "*Lightweight*" represents the lightweight authenticator generation, and "*No Data Loss*" represents whether the compared schemes can protect users from losing data under DFA and SPOF. Besides, the client-side deduplication saves more network bandwidth than the server-side one, and the block-level deduplication saves more storage space than the file-level one.

With regard to data auditing, all the compared schemes can achieve batch auditing. However, only the scheme in [20] and our scheme can achieve efficient public auditing without TPA, which means that they are more practical than [18]. Besides, only [18] and our scheme can share the auditing authenticators and results on the blockchain, to avoid repetitive audit tasks. Note that only our scheme adopts a lightweight authenticator generation algorithm and update protocol to reduce the metadata redundancy caused by authenticators. Considering data deduplication, [20] and our scheme utilize the MLE encryption algorithm to achieve encrypted data deduplication and only our scheme can protect the data users from losing data under DFA or SPOF. Meanwhile, both the property "client-side" and "block-level" of our scheme indicate its effective performance in terms of bandwidth-saving and space-saving. Distinctly, our scheme is more practical than other schemes.

TABLE 2
Notations

Notations	Description
n	Number of blocks in file F
x	Number of challenged blocks in PoWs
y	Number of challenged blocks in auditing
n_d	Number of duplicate blocks of F in SSP
n_a	Number of audit challenge
n_u	Number of users who own file F
H	Evaluation of hash function
E	Evaluation of encryption operation in HCE2
D	Evaluation of decryption operation in HCE2
Exp	Evaluation of exponentiation operation
Pair	Evaluation of bilinear pairing operation
$\|F\|$	Length of the outsourced file F
$\|\mathbb{Z}_p\|$	Length of the element in $\mathbb{Z}_p$
$\|\mathbb{G}\|$	Length of the element in $\mathbb{G}$

5.2 Theoretical Analysis

We further evaluate the compared schemes in terms of computation, communication, and storage costs. For clarity, we define some necessary notations in Table 2. Since the block size of [18], [20] is limited to $|\mathbb{Z}_p|$,[2] we suppose the size of file F is $n|\mathbb{Z}_p|$. According to Ng *et al.*'s work [39], the space-saving efficiency of the deduplication is maximized at block size 4KB. Therefore, we split F into $\omega = \frac{n|\mathbb{Z}_p|}{4KB} \approx \frac{n}{205}$ blocks in our scheme. Then, we summarize the computation, communication, and storage costs of compared schemes in Tables 3 and 4, respectively.

During the initial upload phase, for each data block, all the compared schemes consume 1H+2Exp to calculate authenticator, while [20] and our scheme needs extra 2H +1E to encrypt data block for protecting data privacy. On the whole, the total costs of authenticator generation of our scheme is about $\frac{\omega}{n} \approx \frac{1}{205}$ that of the other two schemes. Concerning communication cost, both [18] and [20] require data users to upload the authenticators and entire file with $n|\mathbb{G}| + 1|F|$, while our scheme only requires data users to upload the authenticators and partial (unique) data blocks. Furthermore, there is no doubt that the communication and storage costs of our scheme will reduce with the increase of block duplicate rate $\frac{n_d}{\omega}$.

In the subsequent upload phase, [20] needs to execute the same workflow as its initial upload process. Instead, [18] and our scheme achieves client-side deduplication by allowing users to upload their data with a valid ownership proof. Therefore, both the computation and communication costs of [18] and our scheme are far lower than that of [20].

In the data auditing process, the total computation costs of our scheme are the same as that of [18], while [20] consumes 2Exp+1Pair to compute extra verification parameters. Moreover, the computation costs of the auditor and auditee (i.e., assignment) in our scheme are different from that of [18], [20] due to our two-way shared auditing mechanism. As regards communication costs, the bandwidth consumption of our scheme is similar to that of [18], while the bandwidth consumed by [20] is almost twice that of our scheme.

Recalling Table 1, only our scheme adopts a lightweight authenticator generation algorithm to reduce the computation and storage costs. However, our scheme has to utilize an authenticator update protocol to alleviate the potential threat that malicious auditee passes audit challenge with pre-computed hash values. As shown in Tables 3 and 4, the costs of our update protocol may be acceptable.

During the data retrieval phase, only [20] and our scheme separately consume n(1D+1H) and ωD to retrieve F from its ciphertext, respectively.

5.3 Implementation

For visualized performance evaluation, we set a series of experiments to test the on-chain and off-chain computation costs of all compared schemes, respectively.

We use Python to implement the above schemes by calling GNU Multiple Precision Arithmetic (GMP) Library[3] and Pairing Based Cryptography (PBC) Library.[4] For each scheme, we estimate the on-chain computation costs, namely gas consumption,[5] of various operations on the official public test-net Ropsten[6] of Ethereum. Furthermore, we also test the off-chain computation costs of data users on a Windows 10 laptop with a 2.30GHz Intel Core i7-1068NG7 CPU and 16GB memory, and the computation costs of SSP on CentOS 8.3.2011 with two 2.20GHz Intel Xeon Silver 4210 CPU and 128GB memory. Specifically, we use SHA-256 and AES-128 to generate an encryption key and encrypt the outsourced data, respectively. Besides, we set the block duplicate rate of tested data as 75%. Notably, all experiments are executed 30 times to obtain an average result.

5.3.1 On-Chain Part Costs

As shown in Fig. 2, we first test the gas costs of creating/modifying the file index table. The scheme in [20] consumes 0 Gwei since it does not need to maintain an index table. Also, the gas costs of [18] are 0.95×10^5Gwei for 1 upload, 2.08×10^5Gwei for 5 uploads, 3.45×10^5Gwei for 10 uploads. Since our scheme only needs to create two index tables in the initial upload phase, the gas costs of our scheme is a constant 1.18×10^5Gwei which is independent of the number of upload processes. No doubt that the gas costs of [18] will far exceed that of our scheme with the growth of user amount in deduplication.

In addition, we also evaluate the gas costs of compared schemes during data auditing phase. According to Ateniese *et al.*'s work [12], when the corrupted rate of outsourced data block is 1%, the precision rates of data auditing technique are 95% during challenging 300 data blocks, and 99% during challenging 460 data blocks. For conciseness, we only test the gas costs of *Challenge*, *prove* and *Verify* operations during challenging 300 and 460 blocks, respectively.

During challenging 300 blocks, the *Challenge* costs of [18], [20] and our scheme are 0.09×10^6Gwei, 3.84×10^6Gwei, and 0.09×10^6Gwei, respectively. Besides, the *Prove* costs of

2. Generally, $|\mathbb{Z}_p|$=160-bits in most existing schemes.

3. https://gmplib.org/

4. https://crypto.stanford.edu/pbc/download.html.

5. Gas is the basic unit used to pay for various computation operations when conducting transactions or executing smart contracts in the Ethereum ecosystem, (1×10^9 Gwei= 1 Ether).

6. https://ropsten.etherscan.io/

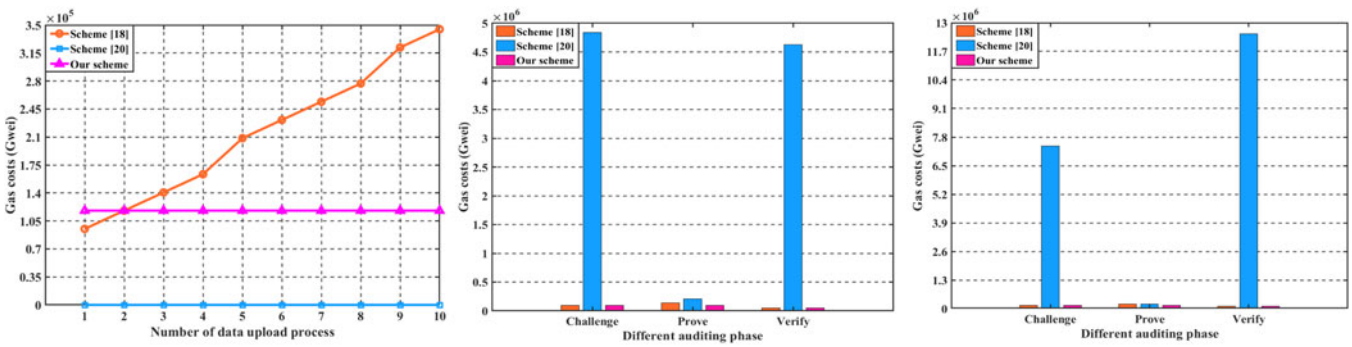
TABLE 3
Comparisons of Computation Costs

Schemes	Initial Upload	Subsequent upload		Data auditing		Authenticator update		Data Retrieval
		Prove	Verify	Prove	Verify	SSP	U	
Scheme [18]	n(1H+2Exp)	Neg	$(2x+1)$Exp $+x$H+2Pair	yExp	$(y+1)$Exp $+y$H+2Pair	—	—	—
Scheme [20]	n(3H+2Exp+1E)	n(3H+2Exp+1E)	Neg	yExp+1Pair	$(y+3)$Exp $+y$H+2Pair	—	—	n(1D+1H)
Our scheme	ω(3H+2Exp+1E)	x(2H+1E)	$(2x+1)$Exp $+x$H+2Pair	Neg	$(2y+1)$Exp $+y$H+2Pair	ω(2H+1Exp) +2Pair	$(\omega+1)$Exp	ωD

Where Neg *represents the computation costs is negligible.*

TABLE 4
Comparisons of Communication and Storage Costs

Schemes	Communication Costs					Storage costs		
	Upload.ini	Upload.sub	Auditing	Update	Retrieval	SSP	U	Blockchain
Scheme [18]	$n\|\mathbb{G}\|+1\|F\|$	$4\|\mathbb{Z}_p\|$	$4\|\mathbb{Z}_p\|$ $+(y+1)\|\mathbb{G}\|$	—	$1\|F\|$	$1\|\mathbb{Z}_p\|+n\|\mathbb{G}\|$ $+1\|F\|$	$2\|\mathbb{Z}_p\|$	$(2n_u+4n_a)\|\mathbb{Z}_p\|$ $+n_a(y+1)\|\mathbb{G}\|$
Scheme [20]	$n\|\mathbb{G}\|+1\|F\|$	$n\|\mathbb{G}\|+1\|F\|$	$2(y+1)\|\mathbb{Z}_p\|$ $+2\|\mathbb{G}\|$	—	$1\|F\|$	$1\|\mathbb{Z}_p\|+n\|\mathbb{G}\|$ $+1\|F\|$	$(2n+3)\|\mathbb{Z}_p\|$	$n_a(y+3)\|\mathbb{Z}_p\|$ $+2n_a\|\mathbb{G}\|$
Our scheme	$\omega(\|\mathbb{Z}_p\|+\|\mathbb{G}\|)$ $+\frac{\omega-n_d}{\omega}\|F\|$	$(x+3)\|\mathbb{Z}_p\|$	$(y+4)\|\mathbb{Z}_p\|$	$2\omega\|\mathbb{G}\|$	$1\|F\|$	$2\|\mathbb{Z}_p\|+\omega\|\mathbb{G}\|$ $+\frac{\omega-n_d}{\omega}\|F\|$	$3\|\mathbb{Z}_p\|$	$(4n_a+2)\|\mathbb{Z}_p\|$



(a) Create/modify index table (b) Auditing challenging 300 blocks (c) Auditing challenging 460 blocks

Fig. 2. Comparisons of on-chain computation costs.

those schemes are 0.14×10^6Gwei, 0.21×10^6Gwei, and 0.09×10^6Gwei, respectively. The *Verify* costs of those schemes are 0.05×10^6Gwei, 4.63×10^6Gwei, and 0.05×10^6Gwei, respectively. During challenging 460 blocks, [18] and our scheme separately consume the same gas costs as they consumed during challenge 300 blocks, while the gas costs of [20] increase to 7.40×10^6Gwei for *Challenge*, 0.21×10^6Gwei for *prove* and 12.49×10^6 for *Verify*. Distinctly, the gas cots of [20] are far higher than other schemes. Considering the current value of Ether, it is uneconomical to use the smart contract techniques to achieve no-TPA public auditing, like [20].

5.3.2 Off-Chain Part Costs

For a clearer explanation, we further analyze the off-chain computation costs of compared schemes from data outsourcing and data auditing phases, respectively.

Data Outsourcing. As is shown in Fig. 3, data outsourcing consists of three phases: initial upload, subsequent upload, and retrieval. Fig. 3a shows the computation costs of compared schemes during the initial upload phase. Recalling Table 1, [18] cannot provide privacy protection for outsourced data. Thus, the time costs 351.75s for 1 MB, 1730.61s for 5MB and 3565.92s for 10MB of [18] are mainly consumed to generate audit authenticators. Furthermore, we can see that the computation cost of [20] is similar to that of [18], which means that the computation cost of data encryption is negligible compared with that of authenticator generation in [20]. Moreover, the costs of our scheme are 1.89s for 1MB, 9.47s for 5MB, and 18.89s for 10MB. Further, we can roughly calculate the costs ratio of our scheme costs to [18], [20]: $\frac{1.89}{351.75} \approx \frac{9.47}{1730.61} \approx \frac{18.89}{3565.92} \approx \frac{1}{190}$, which demonstrates the excellent efficiency of our scheme. Recalling Table 3, $\frac{\omega}{n} = \frac{1}{205}$ and all the compared schemes consume the same costs to compute audit authenticator for each data block. Therefore, we can conclude that in the initial upload phase, our scheme consumed about 92.7% and 7.3% of the total computation cost to compute audit authenticators and encrypted data, respectively.

Due to the server-side deduplication model, the scheme in [20] inevitably consumes the same computation costs as its initial upload process during the subsequent upload phase. However, [18] and our scheme allows data users to upload their data by executing a PoWs protocol with SSP. Distinctly, the computation cost of [20] is much higher than

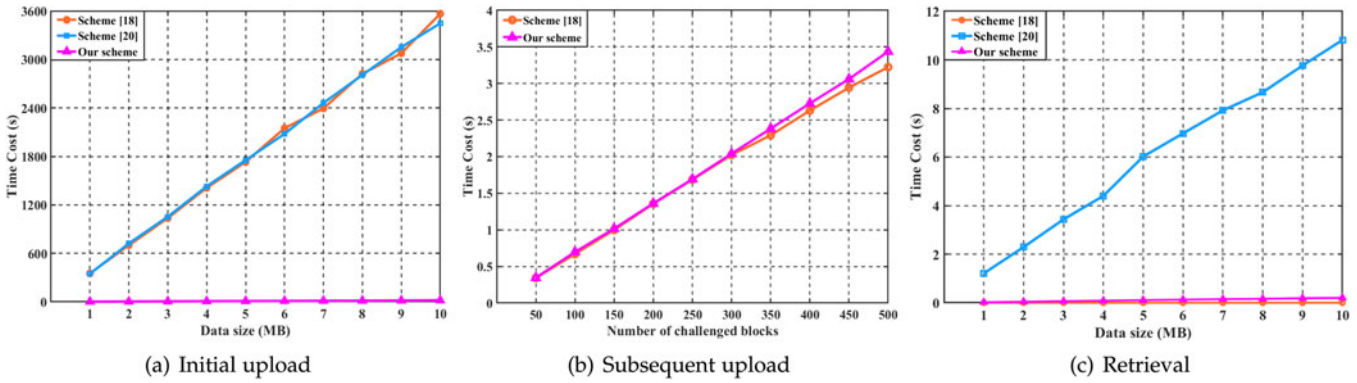


Fig. 3. Comparisons of off-chain computation costs in data outsourcing (deduplication).

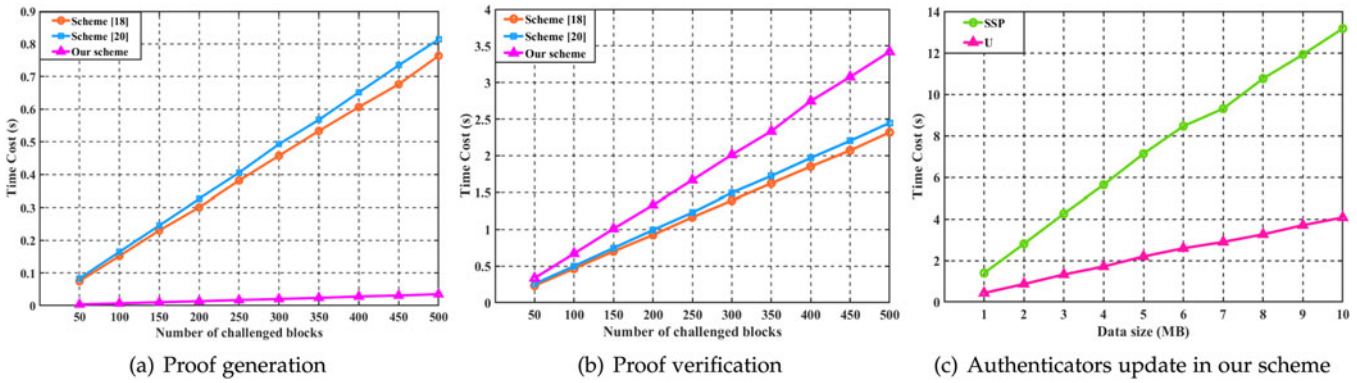


Fig. 4. Comparisons of off-chain computation costs in each data auditing.

that of our scheme and [18]. Therefore, Fig. 3b only demonstrates the costs comparison of our scheme and [18]. Specifically, the costs of [18] are 2.02s for 300 challenged blocks and 2.94s for 460 challenged blocks, while that of our scheme are 2.04s and 3.12s, respectively. Recalling Table 3, we can see that the extra computation costs of our scheme consumed by data encryption $x(2\mathsf{H}+1\mathsf{E})$ are negligible compared with that consumed by proof verification.

Furthermore, Fig. 3c analyzes the computation costs of the compared schemes during the retrieval phase. Due to the plaintext deduplication model, [18] allows data users to directly download the outsourced plaintext. In addition, [20] consumes 1.21s for 1MB, 6.02 for 5MB, and 10.82s for 10MB, while that of our scheme are 0.02s, 0.11s, and 0.21s, respectively. To our knowledge, the impact of data segmentation on the computing cost of AES-128 is negligible. Therefore, [20] consumes the same computation costs as our scheme to encrypt/decrypt the outsourced data, and the extra costs of [20] are consumed to achieve tag checking. Based on the enhanced security mentioned before, our scheme allows data users to execute tag checking only at the necessary situation, and thus saves more computation costs than [20].

Data Auditing. As is demonstrated in Fig. 4, we test the computation costs of the compared schemes during proof generation, proof verification, and authenticator update phases. With regard to proof generation, the computation costs of schemes [18], [20] and our scheme are 0.46s, 0.49s, and 0.02s during challenging 300 blocks, and 0.68s, 0.73s, and 0.03s during challenging 460 blocks. However, the proof verification costs of these schemes are 1.39s, 1.50s, and 2.01s during challenging 300 blocks, and 2.07s, 2.20s, and 3.08s during challenging 460 blocks, respectively.

From Figs. 4a and 4b, we can see that the proof generation costs of our scheme are lower than other schemes, but the proof verification costs are higher than them. The main cause is that our scheme requires the auditee only to generate integrity proof, and ask the auditor aggregate challenged authenticators to verify the integrity proof, while the schemes [18], [20] require the auditee to generate proof and aggregate authenticator and ask the auditor only to verify the proof. In fact, the total costs of our scheme are similar to that of the other compared schemes. In addition, Fig. 4c shows the efficiency of our authenticator update protocol. For 1MB, 5MB, and 10MB data, the computation costs of SSP are 1.40s, 7.13s, and 13.18s, and that of U are 0.43s, 2.20s, and 4.08s, respectively. Evidently, our authenticator update protocol is efficient and available.

To explain the performance of shared auditing, we further test the computation costs of compared schemes during executing repetitive audit tasks launched by different users at the same period, in which the number of challenged blocks in each auditing is 460. As shown in Fig. 5, the proof generation costs of [18], [20], and our scheme in 5 repetitive audit tasks are 0.69s, 3.48s, 0.03s, and the corresponding costs in 10 repetitive audit tasks are 0.69s, 7.01s, 0.03s, respectively. In addition, the proof verification costs of those schemes in 5 repetitive audit tasks are 2.12s, 10.63s, 3.14s, and the corresponding costs in 10 repetitive audit tasks are 2.12s, 21.35s, 3.14s, respectively. Distinctly, [18] and our scheme both achieve shared auditing among data users with the same data, which effectively avoids the resource

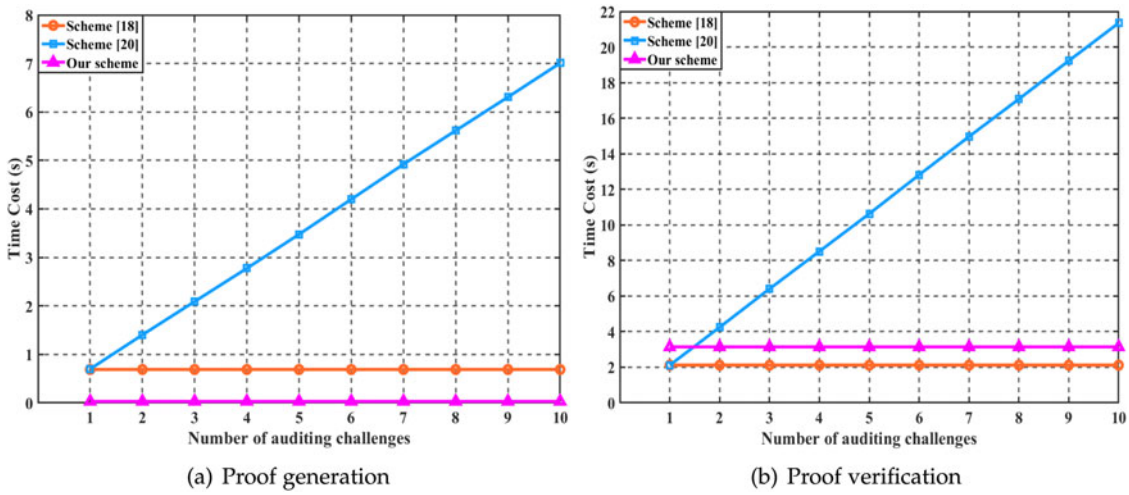


Fig. 5. Comparisons of off-chain computation costs in repetitive data auditing launched by different users.

waste caused by repetitive audit tasks compared with the scheme in [20].

In summary, all experiment results basically satisfy the theoretical analysis of the proposed scheme. Therefore, our scheme has been achieved the desired design goal.

6 Conclusion

Focusing on the availability issues of network storage service, we use the blockchain to establish a double-server storage model and then propose a compact secure deduplication and shared auditing scheme. Specifically, we integrate the double-server storage with a novel deduplication protocol to achieve more efficient space-saving while protecting users from losing data under DFA and SPOF. In addition, we propose a two-way shared auditing mechanism without TPA, where SSPs are endowed with the role of an auditor to check the other SSP who stores the same data, and other users can learn the integrity information of their data without repetitive auditing. On this basis, we design a lightweight authenticator generation algorithm and update protocol to reduce the metadata redundancy caused by authenticators. In conclusion, the proposed scheme is significant in improving the practicability of network storage services.

References

[1] D. Reinsel, J. Gantz, and J. Rydning, "Data age 2025: The digitization of the world from edge to core," Needham, MA, USA, Int. Data Corporation, White Paper, 2018. [Online]. Available: https://www.seagate.com/files/www-content/our-story/trends/files/idc-seagate-dataage-whitepaper.pdf

[2] G. Reinsel and J. Gantz, "The digital universe decade-are you ready?," Needham, MA, USA, Int. Data Corporation, White Paper, 2010. [Online]. Available: https://www.ifap.ru/pr/2010/n100507a.pdf

[3] J. R. Douceur, A. Adya, W. J. Bolosky, P. Simon, and M. Theimer, "Reclaiming space from duplicate files in a serverless distributed file system," in *Proc. 22nd Int. Conf. Distrib. Comput. Syst.*, 2002, pp. 617–624.

[4] J. Paulo and J. Pereira, "A survey and classification of storage deduplication systems," *ACM Comput. Surv.*, vol. 47, no. 1, pp. 1–30, 2014.

[5] M. Bellare, S. Keelveedhi, and T. Ristenpart, "Message-locked encryption and secure deduplication," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, Springer, 2013, pp. 296–312.

[6] J. Li, X. Chen, M. Li, J. Li, P. P. Lee, and W. Lou, "Secure deduplication with efficient and reliable convergent key management," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 6, pp. 1615–1625, Jun. 2014.

[7] R. Chen, Y. Mu, G. Yang, and F. Guo, "Bl-mle: Block-level message-locked encryption for secure large file deduplication," *IEEE Trans. Inf. Forensics Secur.*, vol. 10, no. 12, pp. 2643–2652, Dec. 2015.

[8] Y. Zhao and S. S. Chow, "Updatable block-level message-locked encryption," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, 2017, pp. 449–460.

[9] J. Li, J. Li, D. Xie, and Z. Cai, "Secure auditing and deduplicating data in cloud," *IEEE Trans. Comput.*, vol. 65, no. 8, pp. 2386–2396, Aug. 2016.

[10] F. Zafar *et al.*, "A survey of cloud computing data integrity schemes: Design challenges, taxonomy and future trends," *Comput. Secur.*, vol. 65, pp. 29–49, 2017.

[11] T. Jiang, X. Chen, and J. Ma, "Public integrity auditing for shared dynamic cloud data with group user revocation," *IEEE Trans. Comput.*, vol. 65, no. 8, pp. 2363–2373, Aug. 2016.

[12] G. Ateniese *et al.*, "Provable data possession at untrusted stores," in *Proc. 14th ACM Conf. Comput. Commun. Secur.*, 2007, pp. 598–609.

[13] A. Juels and B. S. Kaliski Jr , "PORs: Proofs of retrievability for large files," in *Proc. 14th ACM Conf. Comput. Commun. Secur.*, 2007, pp. 584–597.

[14] H. Shacham and B. Waters, "Compact proofs of retrievability," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, 2008, pp. 90–107.

[15] Y. Yu, Y. Li, B. Yang, W. Susilo, G. Yang, and J. Bai, "Attribute-based cloud data integrity auditing for secure outsourced storage," *IEEE Trans. Emerging Top. Comput.*, vol. 8, no. 2, pp. 377–390, Apr.–Jun. 2020.

[16] J. Shen, J. Shen, X. Chen, X. Huang, and W. Susilo, "An efficient public auditing protocol with novel dynamic structure for cloud data," *IEEE Trans. Inf. Forensics Secur.*, vol. 12, no. 10, pp. 2402–2415, Oct. 2017.

[17] J. Shen, F. Guo, X. Chen, and W. Susilo, "Secure cloud auditing with efficient ownership transfer," in *Proc. Eur. Symp. Res. Comput. Secur.*, 2020, pp. 611–631.

[18] Y. Xu, C. Zhang, G. Wang, Z. Qin, and Q. Zeng, "A blockchain-enabled deduplicatable data auditing mechanism for network storage services," *IEEE Trans. Emerging Top. Comput.*, vol. 9, no. 3, pp. 1421–1432, Jul.–Sep. 2021.

[19] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. [Online]. Available: https://pdos.csail.mit.edu/6.824/papers/bitcoin.pdf

[20] H. Yuan, X. Chen, J. Wang, J. Yuan, H. Yan, and W. Susilo, "Blockchain-based public auditing and secure deduplication with fair arbitration," *Inf. Sci.*, vol. 541, pp. 409–425, 2020.

[21] G. Wood *et al.*, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum Project Yellow Paper*, vol. 151, no. 2014, pp. 1–32, 2014.

[22] J. Li, X. Chen, F. Xhafa, and L. Barolli, "Secure deduplication storage systems supporting keyword search," *J. Comput. Syst. Sci.*, vol. 81, no. 8, pp. 1532–1541, 2015.

[23] J. Li *et al.*, "Secure distributed deduplication systems with improved reliability," *IEEE Trans. Comput.*, vol. 64, no. 12, pp. 3569–3579, Dec. 2015.

[24] J. Li, J. Wu, L. Chen, and J. Li, "Blockchain-based secure and reliable distributed deduplication scheme," in *Proc. Int. Conf. Algorithms Architectures Parallel Process.*, 2018, pp. 393–405.
[25] S. Halevi, D. Harnik, B. Pinkas, and A. Shulman-Peleg , "Proofs of ownership in remote storage systems," in *Proc. 18th ACM Conf. Comput. Commun. Secur.*, 2011, pp. 491–500.
[26] J. Wang, X. Chen, J. Li, K. Kluczniak, and M. Kutyłowski, "TrDup: enhancing secure data deduplication with user traceability in cloud computing," *I. J. Web Grid Serv.*, vol. 13, no. 3, pp. 270–289, 2017.
[27] K. Kim, T.-Y. Youn, N.-S. Jho, and K.-Y. Chang, "Client-side deduplication to enhance security and reduce communication costs," *Etri J.*, vol. 39, no. 1, pp. 116–123, 2017.
[28] G. Tian, H. Ma, Y. Xie, and Z. Liu, "Randomized deduplication with ownership management and data sharing in cloud storage," *J. Inf. Secur. Appl.*, vol. 51, no. 102432, pp. 1–9, Apr. 2020.
[29] Y. Deswarte, J.-J. Quisquater, and A. Saïdane, "Remote integrity checking," in *Proc. Work. Conf. Integrity Intern. Control Inf. Syst.*, 2003, pp. 1–11.
[30] Q. Wang, C. Wang, K. Ren, W. Lou, and J. Li, "Enabling public auditability and data dynamics for storage security in cloud computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 22, no. 5, pp. 847–859, May 2011.
[31] D. Boneh, B. Lynn, and H. Shacham, "Short signatures from the weil pairing," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, 2001, pp. 514–532.
[32] Q. Zheng and S. Xu, "Secure and efficient proof of storage with deduplication," in *Proc. 2nd ACM Conf. Data Appl. Secur. Privacy*, 2012, pp. 1–12.
[33] J. Yuan and S. Yu, "Secure and constant cost public cloud storage auditing with deduplication," in *Proc. IEEE Conf. Commun. Netw. Secur.*, 2013, pp. 145–153.
[34] X. Liu, W. Sun, W. Lou, Q. Pei, and Y. Zhang, "One-tag checker: Message-locked integrity auditing on encrypted cloud deduplication storage," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, 2017, pp. 1–9.
[35] H. Huang, X. Chen, Q. Wu, X. Huang, and J. Shen, "Bitcoin-based fair payments for outsourcing computations of fog devices," *Future Gener. Comput. Syst.*, vol. 78, no. 2, pp. 850–858, Jan. 2018.
[36] H. Huang, X. Chen, and J. Wang, "Blockchain-based multiple groups data sharing with anonymity and traceability," *Sci. China Inf. Sci.*, vol. 63, no. 3, pp. 1–13, 2020.
[37] Y. Zhang, C. Xu, X. Lin, and X. S. Shen, "Blockchain-based public integrity verification for cloud storage against procrastinating auditors," *IEEE Trans. Cloud Comput.*, vol. 9, no. 3, pp. 923–937, Jul.–Sep. 2021.
[38] Y. Xu, J. Ren, Y. Zhang, C. Zhang, B. Shen, and Y. Zhang, "Blockchain empowered arbitrable data auditing scheme for network storage as a service," *IEEE Trans. Serv. Comput.*, vol. 13, no. 2, pp. 289–300, Mar./Apr. 2020.
[39] C.-H. Ng and P. P. Lee, "Revdedup: A reverse deduplication storage system optimized for reads to latest backups," in *Proc. 4th Asia-Pacific Workshop Syst.*, 2013, pp. 1–7.

Guohua Tian received the BS degree from the School of Mathematics and Information Science, Shaanxi Normal University, in 2016 and the MS degree in 2019 from the School Mathematics and Statistics, Xidian University, where he is currently working toward the PhD degree in cyberspace security with the School of Cyber Engineering. His research interests include network and information security, cloud computing, data deduplication, and blockchain.

Yunhan Hu received the BE degree from the School of Computer Science and Technology, Chongqing University of Post and Telecommunications, in 2019. He is currently working toward the master's degree in cyberspace security with the School of Cyber Engineering, Xidian University. His research interests include network and information security and blockchain.

Jianghong Wei received the PhD degree in information security from Zhengzhou Information Science and Technology Institute, Zhengzhou, China, in 2016. He is currently a lecturer with the State Key Laboratory of Mathematical Engineering and Advanced Computing, Zhengzhou, China. His research interests include applied cryptography and network security.

Zheli Liu received the BSc and MSc degrees in computer science and PhD degree in computer application from Jilin University, China, in 2002, 2005, and 2009, respectively. After a postdoctoral fellowship with Nankai University, he joined the College of Cyber Science, Nankai University, in 2011. He is currently a professor with Nankai University. His research interests include applied cryptography and data privacy protection.

Xinyi Huang received the PhD degree from the School of Computer Science and Software Engineering, University of Wollongong, Australia. He is currently a professor with the College of Mathematics and Informatics, Fujian Normal University, China. He has authored more than 150 research papers in refereed international conferences and journals. His research interests include applied cryptography and network security.

Xiaofeng Chen (Senior Member, IEEE) received the BS and MS degrees in mathematics from Northwest University, China, in 1998 and 2000, respectively, and the PhD degree in cryptography from Xidian University in 2003. He is currently a professor with Xidian University. He has authored or coauthored more than 200 research papers in refereed international conferences and journals. His research interests include applied cryptography and cloud computing security. His work has been cited more than 10000 times at Google Scholar. He is on the editorial board of *IEEE Transactions on Dependable and Secure Computing*, *Security and Privacy*, *Computer Standards & Interfaces*, and *Computing and Informatics*. He has served as the program or the general chair or a program committee member in more than 30 international conferences.

Willy Susilo (Fellow) received the PhD degree in computer science from the University of Wollongong, Australia. He is currently a distinguished professor, the head of the School of Computing and Information Technology, and the director of the Institute of Cyber security and Cryptology, University of Wollongong. He has authored or coauthored numerous publications in the area of digital signature schemes and encryption schemes. His research interests include cryptography and information security. He was a program committee member in dozens of international conferences. He was the recipient of prestigious Australian Research Council (ARC) Future Fellow.